Practical 7: Tree and Graph Data Structures with DFS and BFS

Aim

To study and implement graph traversal techniques - Breadth-First Search (BFS) and Depth-First Search (DFS).

Theory

A Tree is a hierarchical, acyclic connected graph with a single root node, where every node (except the root) has exactly one parent, commonly used to represent hierarchical relationships such as file systems and organization charts. A Graph is a more general data structure consisting of nodes (vertices) connected by edges, which may be directed or undirected, weighted or unweighted, and can contain cycles; it is often represented using an adjacency list (a dictionary mapping each node to its neighbors, as used here) or an adjacency matrix. Breadth-First Search (BFS) explores a graph level by level using a queue (FIFO), visiting all neighbors of the current node before moving to the next level, which makes it well suited for finding the shortest path in an unweighted graph. Depth-First Search (DFS) explores as far as possible along each branch before backtracking, implemented either recursively (using the call stack) or iteratively with an explicit stack, and is useful for tasks like cycle detection and topological sorting. Both algorithms maintain a 'visited' set to avoid revisiting nodes and getting stuck in infinite loops when cycles are present, and both run in O(V + E) time, where V is the number of vertices and E is the number of edges.

Algorithm

1. Represent the graph as a dictionary of adjacency lists.

2. For BFS: initialize a visited set and a queue with the start node; repeatedly dequeue a node, record it, and enqueue its unvisited neighbors.

3. For DFS: recursively visit the start node, mark it visited, record it, and recurse into its unvisited neighbors.

4. Call bfs() and dfs() from the same starting node and print the traversal order for each.

Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': []
}

def bfs(start):
    visited, queue, order = {start}, [start], []
    while queue:
        node = queue.pop(0)
        order.append(node)
        for n in graph[node]:
            if n not in visited:
                visited.add(n)
                queue.append(n)
    return order

def dfs(node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for n in graph[node]:
        if n not in visited:
            dfs(n, visited, order)
    return order
print("BFS Traversal:", bfs('A'))
```

```
print("DFS Traversal:", dfs('A'))
```

Output

```
BFS Traversal: ['A', 'B', 'C', 'D']
DFS Traversal: ['A', 'B', 'D', 'C']
```

Result / Conclusion

BFS and DFS graph traversal algorithms were successfully implemented and their traversal orders were verified.

Viva Questions

Q1. What is the key difference between BFS and DFS?

Ans: BFS (Breadth-First Search) uses a queue (FIFO) and explores all neighbors of a node before going deeper — it visits level by level. DFS (Depth-First Search) uses a stack (or recursion) and explores as far down one branch as possible before backtracking. BFS finds the shortest path in an unweighted graph; DFS is better for tasks like cycle detection, topological sorting, and solving mazes.

Q2. What is an adjacency list and how does it compare to an adjacency matrix?

Ans: An adjacency list stores each vertex's neighbors as a list/set (e.g., a dictionary). Space: O(V + E). Efficient for sparse graphs. An adjacency matrix uses a V×V grid where matrix[i][j]=1 if edge (i,j) exists. Space: O(V^2). Efficient for dense graphs and O(1) edge lookup. For most real-world graphs (sparse), adjacency lists are preferred.

Q3. Why is a 'visited' set necessary in graph traversal?

Ans: Unlike trees, graphs can contain cycles (e.g., A→B→C→A). Without tracking visited nodes, the traversal would revisit nodes indefinitely, causing an infinite loop. The visited set records nodes already processed so they are skipped on re-encounter. For trees (which are acyclic), a visited set is not strictly necessary but is good practice.

Q4. What is the time complexity of BFS and DFS?

Ans: Both BFS and DFS have time complexity O(V + E), where V is the number of vertices and E is the number of edges. Each vertex is visited once (O(V)) and each edge is examined once (O(E)). Space complexity: BFS uses O(V) for the queue (all nodes at the widest level); DFS uses O(V) for the recursion call stack (depth of the graph in the worst case).

Q5. Give a real-world application of BFS and DFS each.

Ans: BFS applications: shortest path in unweighted graphs (GPS navigation, social network friend suggestions at distance 1, 2, 3), level-order traversal of a tree, web crawlers. DFS applications: detecting cycles in a graph, topological sorting (dependency resolution), solving puzzles like mazes and Sudoku, finding connected components, generating permutations.